JAVA SERVLET

Jakarta Servlet

1. Aim

To study server-side programming with Java Servlet and implement a web application covering HTTP, Servlet API, Servlet life cycle, configuration and context, request/response objects, session handling, Servlet events and filters.

2. Syllabus Covered

- Server-side programming with Java Servlet
- HTTP and Servlet
- Servlet API
- Servlet life cycle
- Servlet configuration and context
- Request and Response objects
- Session handling
- Event handling
- Introduction to filters and a simple filter application

3. Learning Objectives

- Explain Servlet and Servlet Container.
- Create and map an HttpServlet.
- Handle GET and POST requests.
- Read form parameters and generate responses.
- Explain init(), service(), doGet(), doPost() and destroy().
- Differentiate ServletConfig and ServletContext.
- Use Cookie and HttpSession.
- Understand listeners/events.
- Create a simple Filter and use FilterChain.

4. Server-Side Programming

Server-side programming executes application logic on the server. The browser sends an HTTP request; the server processes it and returns an HTTP response.

```
Browser
   ↓ HTTP Request
Apache Tomcat
   ↓
Servlet
   ↓ Java Processing
HTTP Response
   ↓
Browser
```

5. What is a Servlet?

A Servlet is a Java class that runs inside a Servlet container and processes client requests, usually HTTP requests, to generate dynamic responses. Apache Tomcat provides the Servlet container.

6. Why Servlets?

- Generate dynamic web content.
- Process HTML form data.
- Implement server-side business logic.
- Connect application logic to databases through JDBC.
- Manage sessions and cookies.
- Support authentication, authorization and request processing.

7. Servlet Architecture

```
                Browser
                   |
            HTTP Request
                   ↓
          Apache Tomcat
          Servlet Container
                   |
                   ↓
               Servlet
                   |
                   ↓
           Business Logic
                   |
                   ↓
                JDBC
                   |
                   ↓
             Database
                   |
                   ↓
            HTTP Response
                   |
                   ↓
               Browser
```

8. HTTP and Servlet

HTTP is the protocol commonly used for communication between browsers and web servers. The two methods emphasized in this experiment are GET and POST.

```
GET  → doGet()
POST → doPost()
```

- GET is commonly used to request/read data.
- POST is commonly used to submit data for processing.

9. Servlet API

For Tomcat 11, use the Jakarta Servlet API.

```
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
```

Class / Interface	Purpose	Example
HttpServlet	Base class for HTTP Servlets	extends HttpServlet
HttpServletRequest	Incoming request data	getParameter()
HttpServletResponse	Outgoing response	getWriter()
ServletConfig	Servlet-specific configuration	getServletConfig()
ServletContext	Application-wide context	getServletContext()
HttpSession	Server-side session state	setAttribute()
Cookie	Client-associated data	addCookie()
Filter	Intercepts requests/responses	doFilter()

10. Maven Configuration for Tomcat 11

```
<packaging>war</packaging>

<dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>6.1.0</version>
    <scope>provided</scope>
</dependency>
```

WAR packaging is used for web deployment. The provided scope means Tomcat supplies the Servlet API at runtime.

11. Project Structure

```
servlet/
├── pom.xml
└── src/main/
    ├── java/com/servlet/
    │   ├── HelloServlet.java
    │   ├── LoginServlet.java
    │   ├── HomeServlet.java
    │   ├── AuthFilter.java
    │   └── AppListener.java
    └── webapp/
        └── index.html
```

12. First Servlet: HelloServlet

```
package com.servlet;

import java.io.IOException;
import java.io.PrintWriter;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
```

```
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        out.println("<h1>Hello Students!</h1>");
        out.println("<h2>Welcome to Servlet Programming</h2>");
    }
}
```

13. URL Mapping

@WebServlet maps a Servlet to a URL pattern.

```
@WebServlet("/hello")

http://localhost:8080/servlet/hello

localhost = local computer
8080      = Tomcat port
servlet   = application/context path
hello     = Servlet mapping
```

14. Request and Response Objects

HttpServletRequest represents information received from the client.

```
String name = request.getParameter("name");
String method = request.getMethod();
String uri = request.getRequestURI();
```

HttpServletResponse is used to construct the server response.

```
response.setContentType("text/html");
PrintWriter out = response.getWriter();
out.println("<h1>Welcome</h1>");
```

15. GET Form

```
<!DOCTYPE html>
<html>
<body>
<h1>Student Information</h1>
<form action="hello" method="get">
    Enter Name:
    <input type="text" name="name">
    <input type="submit" value="Submit">
</form>
```

```
</body>
</html>

String name = request.getParameter("name");
out.println("<h1>Welcome " + name + "</h1>");
```

A GET request may produce a URL such as:

```
http://localhost:8080/servlet/hello?name=Jahanvi
```

16. POST Form

```
<form action="hello" method="post">
    Enter Name:
    <input type="text" name="name">
    <input type="submit" value="Submit">
</form>

@Override
protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException {

    response.setContentType("text/html");
    String name = request.getParameter("name");

    PrintWriter out = response.getWriter();
    out.println("<h1>Welcome " + name + "</h1>");
}
```

17. GET vs POST

Feature	GET	POST
Servlet method	doGet()	doPost()
Typical use	Request/read data	Submit/process data
Parameter visibility	Query data can appear in URL	Data is sent in request body

18. Servlet Life Cycle

```
Servlet class
    ↓
Loading
    ↓
Instantiation
    ↓
init()       ← initialization
    ↓
service()    ← request dispatch
    ↓
doGet()/doPost()
    ↓
destroy()    ← cleanup
```

The container controls the lifecycle; application code does not normally call these lifecycle methods directly.

19. Life Cycle Methods

- init() — called when the Servlet is initialized, normally once for an instance.
- service() — receives requests and dispatches them to HTTP methods.
- doGet() — handles GET requests.
- doPost() — handles POST requests.
- destroy() — called when the Servlet is taken out of service.

```
public class LifeCycleServlet extends HttpServlet {

    @Override
    public void init() {
        System.out.println("Servlet Initialized");
    }

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html");
        response.getWriter().println("Request Processed");
    }

    @Override
    public void destroy() {
        System.out.println("Servlet Destroyed");
    }
}
```

20. ServletConfig

ServletConfig contains configuration information specific to a particular Servlet.

```
ServletConfig config = getServletConfig();
String value = config.getInitParameter("name");
```

21. ServletContext

ServletContext represents the web application as a whole and can be used for application-wide information/resources.

```
ServletContext context = getServletContext();
```

22. ServletConfig vs ServletContext

Feature	ServletConfig	ServletContext
Scope	One Servlet	Entire application
Purpose	Servlet-specific configuration	Application-wide information/resources
Access	getServletConfig()	getServletContext()

23. Session Handling

HTTP is stateless: separate requests do not automatically carry application-level identity. Session tracking associates requests with a user/session.

- Cookies
- HttpSession
- URL rewriting (conceptual)

24. Cookies

A Cookie is a small client-associated value that can be sent with subsequent requests.

```
Cookie cookie = new Cookie("username", "Jahanvi");
response.addCookie(cookie);
```

Reading cookies:

```
Cookie[] cookies = request.getCookies();

if (cookies != null) {
    for (Cookie cookie : cookies) {
        if ("username".equals(cookie.getName())) {
            String username = cookie.getValue();
        }
    }
}
```

25. HttpSession

HttpSession provides server-side session state associated with a client session.

```
HttpSession session = request.getSession();
session.setAttribute("username", "Jahanvi");

String username =
    (String) session.getAttribute("username");

session.invalidate();
```

26. Session Flow

```
Login
  ↓
Create / obtain session
  ↓
Store username
  ↓
Access protected resource
  ↓
Read session attribute
  ↓
Logout
  ↓
session.invalidate()
```

27. Cookie vs HttpSession

Feature	Cookie	HttpSession
Typical storage	Client-associated	Server-side session data
Servlet API	Cookie	HttpSession
Example	username cookie	logged-in user information

28. Servlet Event Handling

Listeners are components that receive notifications when specified application or session events occur.

- ServletContextListener — application context initialization/destruction.
- HttpSessionListener — session creation/destruction.

29. ServletContextListener Example

```
import jakarta.servlet.ServletContextEvent;
import jakarta.servlet.ServletContextListener;
import jakarta.servlet.annotation.WebListener;

@WebListener
public class AppListener
        implements ServletContextListener {

    @Override
    public void contextInitialized(
            ServletContextEvent event) {
        System.out.println("Application Started");
    }

    @Override
    public void contextDestroyed(
            ServletContextEvent event) {
        System.out.println("Application Stopped");
    }
}
```

30. HttpSessionListener Example

```
@WebListener
public class SessionListener
        implements HttpSessionListener {

    @Override
    public void sessionCreated(
            HttpSessionEvent event) {
        System.out.println("Session Created");
    }

    @Override
    public void sessionDestroyed(
            HttpSessionEvent event) {
        System.out.println("Session Destroyed");
    }
}
```

31. What is a Filter?

A Filter intercepts requests and responses around a target resource such as a Servlet. Filters are useful for common processing that should happen across selected requests.

```
Browser
   ↓ Request
 Filter
   ↓
 Servlet
   ↓ Response
 Filter
   ↓
Browser
```

- Authentication/authorization
- Logging
- Validation
- Security checks
- Character encoding
- Request preprocessing and response postprocessing

32. Simple Filter Application

```
package com.servlet;

import java.io.IOException;
import jakarta.servlet.Filter;
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.ServletRequest;
import jakarta.servlet.ServletResponse;
import jakarta.servlet.annotation.WebFilter;

@WebFilter("/*")
public class MyFilter implements Filter {

    @Override
    public void doFilter(
            ServletRequest request,
            ServletResponse response,
            FilterChain chain)
            throws IOException, ServletException {

        System.out.println("Request intercepted");

        chain.doFilter(request, response);

        System.out.println("Response completed");
    }
}
```

chain.doFilter(request, response) passes processing to the next component in the filter chain.

33. Integrated Practical: Student Login + Session + Filter

This application combines the major practical Servlet concepts into one small application.

33.1 Login Form

```
<!DOCTYPE html>
<html>
<body>
<h1>Student Login</h1>

<form action="login" method="post">
    Username:
    <input type="text" name="username">
    <br><br>

    Password:
    <input type="password" name="password">
    <br><br>

    <input type="submit" value="Login">
</form>

</body>
</html>
```

33.2 LoginServlet

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        PrintWriter out = response.getWriter();

        if ("admin".equals(username)
                && "1234".equals(password)) {

            HttpSession session = request.getSession();
            session.setAttribute("username", username);

            out.println("<h1>Login Successful</h1>");
            out.println("<a href='home'>Go to Home</a>");

        } else {
            out.println("<h1>Invalid Login</h1>");
        }
    }
}
```

33.3 Authentication Filter

```
@WebFilter("/home")
public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request,
                         ServletResponse response,
```

```
                            FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest req =
            (HttpServletRequest) request;

        HttpServletResponse res =
            (HttpServletResponse) response;

        HttpSession session =
            req.getSession(false);

        if (session != null &&
            session.getAttribute("username") != null) {

            chain.doFilter(request, response);

        } else {
            res.sendRedirect("index.html");
        }
    }
}
```

33.4 HomeServlet

```
@WebServlet("/home")
public class HomeServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        HttpSession session =
            request.getSession(false);

        String username =
            (String) session.getAttribute("username");

        PrintWriter out = response.getWriter();

        out.println("<h1>Student Home</h1>");
        out.println("<h2>Welcome " + username + "</h2>");
    }
}
```

34. Integrated Flow

```
index.html
    ↓ POST
LoginServlet
    ↓
Validate credentials
    ↓
Create HttpSession
    ↓
/home
    ↓
```

```
AuthFilter
    ↓
Session valid?
  /       \
YES       NO
 |         |
Home     Redirect
Servlet  to login
```

35. Build and Deploy

- Open the Maven project in VS Code.
- Run mvn package.
- Find target/servlet.war.
- Copy servlet.war into Tomcat/webapps.
- Start Tomcat using startup.bat.
- Open the deployed application in the browser.

```
mvn package

http://localhost:8080/servlet/
```

36. Complete Request Processing Flow

```
Browser
   ↓
HTTP Request
   ↓
Tomcat
   ↓
Filter (if mapped)
   ↓
Servlet Container
   ↓
Servlet
   ↓
Request Processing
   ↓
HTTP Response
   ↓
Filter
   ↓
Browser
```

37. Common Errors

Problem	Check
404 Not Found	WAR deployment, context path, URL mapping and Tomcat status.
500 Server Error	Java code, exception details and Tomcat logs.
javax.servlet error	Use jakarta.servlet.* with Tomcat 11.
Form parameter is null	Check input name and request.getParameter() name.
Session value is null	Check session creation and attribute name.
Filter blocks request	Call chain.doFilter() when the request should continue.
Port 8080 busy	Stop the conflicting process or change the Tomcat port.

38. Practical Tasks

- Create HelloServlet using doGet().
- Create a Student Form using POST.
- Display name, email and course using getParameter().
- Store username in HttpSession.
- Create logout using session.invalidate().
- Create a ServletContextListener for application start/stop.
- Create a Filter that logs request URIs.
- Create an authentication Filter for a protected Servlet.

39. Viva Questions

Q. What is a Servlet?

Ans. A Java server-side class managed by a Servlet container to process requests and generate responses.

Q. What is a Servlet Container?

Ans. A runtime environment that manages Servlet lifecycle and request processing.

Q. What is Tomcat?

Ans. A web server and Servlet container used to run Java web applications.

Q. What is HttpServlet?

Ans. A base class commonly extended to create HTTP Servlets.

Q. What is @WebServlet?

Ans. An annotation used to map a Servlet to a URL pattern.

Q. What is doGet()?

Ans. It handles HTTP GET requests.

Q. What is doPost()?

Ans. It handles HTTP POST requests.

Q. What is Servlet lifecycle?

Ans. Loading/creation, initialization, request processing and destruction managed by the container.

Q. What is ServletConfig?

Ans. Configuration information specific to one Servlet.

Q. What is ServletContext?

Ans. Context representing the entire web application.

Q. What is HttpServletRequest?

Ans. Object representing the incoming client request.

Q. What is HttpServletResponse?

Ans. Object used to construct the server response.

Q. Why is session tracking needed?

Ans. HTTP is stateless, so session mechanisms associate multiple requests with a user/session.

Q. What is HttpSession?

Ans. A server-side mechanism for storing session-associated information.

Q. What is a Cookie?

Ans. A small client-associated value sent according to cookie rules.

Q. What is a Listener?

Ans. A component that receives notifications about specified application or session events.

Q. What is a Filter?

Ans. A component that intercepts requests/responses around a target resource.

Q. Why is chain.doFilter() used?

Ans. To pass processing to the next component in the filter chain.

Q. Which package is used with Tomcat 11?

Ans. jakarta.servlet and jakarta.servlet.http.

40. Key Points for Examination

- Servlet = server-side Java web component.
- Tomcat = Servlet container/web server environment.
- GET → doGet(); POST → doPost().
- Request data → HttpServletRequest.
- Response → HttpServletResponse.
- Form data → request.getParameter().
- Servlet-specific configuration → ServletConfig.
- Application-wide context → ServletContext.
- Session state → HttpSession.
- Client-associated value → Cookie.
- Events → Listeners.
- Request/response interception → Filter.
- Continue filter chain → chain.doFilter().
- Tomcat 11 → Jakarta Servlet namespace.